Technical Analysis of Workflow Segmentation

From Rule-Based Temporal Segmentation to a Learned AI System

Charan Rongali
Indian Institute of Technology Hyderabad
Department of Electrical Engineering

Abstract

This section documents the technical reasoning behind the workflow-segmentation solution developed for the AI Engineer intern task. The final system is not a neural network; it is an interpretable temporal segmentation pipeline based on event timing, activity density, application/context transitions, screenshots and task evidence. The important engineering question is therefore not whether a neural network was used, but why it was not the first choice, where the current system is computationally efficient, where it is limited, and how the same problem could be converted into a learned temporal model when more labelled data becomes available.

1 What the Final System Actually Is

The final solution is best described as an:

Interpretable Temporal Workflow Segmentation System

It takes a chronological event stream and identifies intervals that are likely to correspond to meaningful user tasks.

Let

$$\mathcal{E} = \{e_1, e_2, \ldots, e_N\}$$

be the events in one session. Each event can be represented as

$$e_i = (t_i, a_i, w_i, p_i, x_i),$$

where t_i is the timestamp, a_i is the active application, w_i is contextual/window information, p_i is the event/payload information and x_i is extracted textual evidence when available.

The output is a set of non-overlapping workflow intervals

$$\mathcal{S} = \{S_1, S_2, \ldots, S_M\}, \qquad S_j = [s_j, e_j].$$

The system is not trained using gradient descent. Instead, it combines observable signals and deterministic thresholds to decide where a workflow is likely to begin and end.

This was a deliberate engineering choice because the available labelled data was limited and the boundary problem was highly interpretable from temporal and application-level signals.

2 Why This Is Not a Neural Network

A neural network would learn a parameterized function

$$f_\theta(X)$$

where θ represents trainable parameters.

For example, a neural temporal model could contain layers of the form

$$h^{(l+1)} = \sigma\left(W^{(l)} h^{(l)} + b^{(l)}\right),$$

and would normally be trained by minimizing an objective such as

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(X_i), Y_i) + \lambda R(\theta).$$

None of these trainable components are present in the final implementation.

Therefore:

Component	Final system
Trainable neural layers	None
Hidden layers	None
Learnable weights	None
Backpropagation	None
Loss function for training	Not applicable
Optimizer	Not applicable
Dropout	Not used
Batch normalization	Not used
Weight decay	Not used
Skip/residual connections	Not applicable
GPU training	Not required

This should not be presented as a weakness. The correct technical description is that the current system is a strong, interpretable baseline for a temporal segmentation problem.

3 Why a Rule-Based Baseline Was Reasonable

There were three practical reasons.

3.1 Limited supervised data

Dataset A contained 2,009 ground-truth execution intervals and Dataset B produced 173 final segments. This is useful for analysis, but it is not a large supervised dataset for training a high-capacity sequence model.

A neural network trained on a small number of sessions could memorize session-specific patterns rather than learn general workflow boundaries.

3.2 The signals were already meaningful

The raw logs directly contained:

- timestamps,
- application changes,
- browser navigation,
- mouse activity,
- keyboard activity,
- screenshots,
- shortcuts,
- clipboard activity,
- extracted task text.

These signals have direct temporal interpretations. It was therefore possible to build an interpretable baseline before introducing a learned model.

3.3 Automation requires explainability

If a downstream system is going to perform an action automatically, it is useful to know why a workflow boundary was detected.

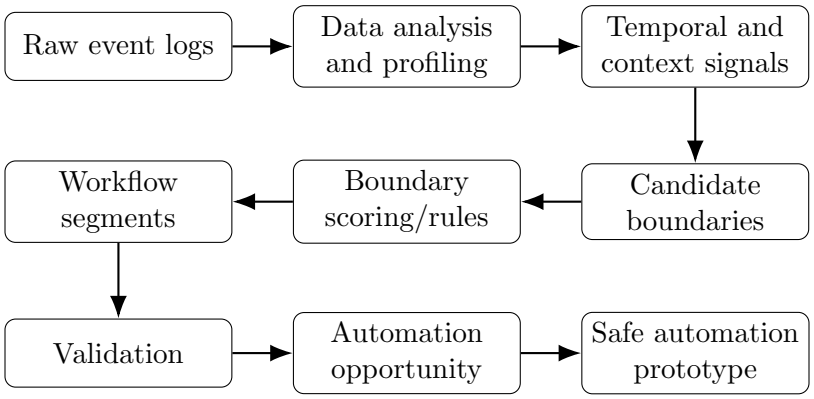
For example:

$$\text{Boundary confidence} \leftarrow \begin{cases} \text{strong gap} \\ \text{application transition} \\ \text{activity burst} \\ \text{screenshot checkpoint} \\ \text{task evidence} \end{cases}$$

is easier to audit than an unexplained probability.

4 Complete Technical Workflow

The final workflow can be summarized as:



The key engineering decision is that the system does not jump directly from raw events to automation. There is a validation stage between segmentation and downstream automation.

5 Dataset Analysis and What Failed

5.1 Application-switch-only segmentation

Dataset A contained 50,588 application-switch events. However, only 7,380 represented an actual change in application identity.

More importantly, application transitions did not always coincide with ground-truth execution starts.

The observed distances from transitions to ground-truth starts showed that:

1171

transitions were within 0.5 seconds,

1237

were within 2 seconds, and

1457

were within 5 seconds.

Therefore, application changes were informative but not sufficient.

5.2 Screenshot-only segmentation

There were 1,905 ground-truth starts whose immediately preceding relevant event was a screenshot.

This made screenshots valuable as boundary context. However, a screenshot is a checkpoint rather than an explicit annotation saying "a new task starts here."

Consequently, screenshots were used as contextual evidence rather than as a hard segmentation rule.

5.3 Gap-only segmentation

The raw event gap distribution was highly concentrated around short intervals:

$$\text{median}(\Delta t) \approx 0.057s,$$

while the mean was approximately

$$0.655s.$$

Long gaps existed but were relatively rare. Therefore, gap thresholds could help identify boundaries, but a gap alone could also split one long workflow incorrectly.

5.4 Completion-as-start failure

Dataset B provided a particularly clear failure.

A completion-centred segmentation version produced:

173

segments with an average duration of approximately

$$4.87s.$$

This was too short to represent the complete user interaction in many cases.

The important correction was:

$$\boxed{\text{completion evidence} \neq \text{task-start evidence}}$$

The V8 system therefore searches backward from a reliable completion anchor for an earlier activity block.

6 Dataset A Version Evolution

The main development progression was:

Version	Main idea	Observation
V1–V3	Initial event/activity segmentation	Useful for discovering basic event patterns but produced weak boundaries.
V4	Stronger activity episodes and gaps	Precision/recall remained low.
V5–V6	Checkpoints, activity transitions and gap logic	Recall improved substantially.
V7	Combined activity, transitions, checkpoints and maturity	Best overall balance of precision, recall and F1.
V8	Additional boundary refinement	Slightly better IoU but worse overall F1 than V7.

The final Dataset A version was V7.

$$P = 46.47\%, \qquad R = 55.95\%, \qquad F_1 = 50.77\%.$$

The mean IoU was

$$66.37\%.$$

Mean start and end errors were

$$5.94s \quad \text{and} \quad 6.96s.$$

7 Dataset A Mathematical Evaluation

For predicted segment P and ground-truth segment G:

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|}.$$

A prediction is considered a match when

$$IoU(P, G) \geq 0.5$$

under the one-to-one matching procedure.

Boundary errors are

$$E_s = |s_P - s_G|, \qquad E_e = |e_P - e_G|.$$

Precision, recall and F1 are:

$$P = \frac{TP}{TP + FP},$$

$$R = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}.$$

This evaluation was performed after prediction generation. The ground-truth intervals were not used as input to the final segmentation algorithm.

8 Dataset B V8 Boundary Logic

For each candidate boundary c, the system considers a local temporal context.

The main quantities are:

$$g(c) = \text{local temporal gap},$$

$$a(c) = \text{activity density after the candidate},$$

$$d(c) = \text{application/context change},$$

$$x(c) = \text{task-related evidence},$$

$$q(c) = \text{task/completion consistency}.$$

A conceptual boundary score can be written as

$$S(c) = w_g g(c) + w_a a(c) + w_d d(c) + w_x x(c) + w_q q(c).$$

The implementation is deterministic rather than learned; the equation is a mathematical description of how the evidence can be viewed conceptually.

The principal V8 parameters were:

$$L = 60s$$

maximum backward lookback,

$$\tau_s = 5s$$

strong gap,

$$\tau_{vs} = 8s$$

very strong gap,

$$T_{\max} = 120s$$

maximum segment duration.

The system also required subsequent activity before accepting a candidate start.

9 V8 Results

Dataset B V8 produced:

$$173$$

segments.

The duration statistics were:

$$\bar{T} = 32.63s,$$

$$\text{median}(T) = 30.90s,$$

$$T_{\min} = 1.19s, \qquad T_{\max} = 61.57s.$$

The total observed segment time was:

$$94.09 \text{ minutes.}$$

The start detection methods were:

$$165 \text{ activity-block boundaries}$$

and

$$8 \text{ fallback starts.}$$

Confidence was:

$$56 \text{ high}, \quad 109 \text{ medium}, \quad 8 \text{ low.}$$

No overlapping V8 segment pairs were found.

10 Algorithm Pseudocode

Algorithm 1 Dataset B V8 Workflow Segmentation

Require: Chronologically ordered events $\mathcal{E}$, maximum lookback L
Ensure: Non-overlapping workflow segments

```
Identify strong completion/task anchors
Set used_until ← −1
for each completion anchor c do
    Search backward up to L = 60s
    Generate candidate activity-block boundaries
    for each candidate b do
        Compute temporal-gap evidence
        Compute activity-density evidence
        Compute application/context transition evidence
        Compute task-text/context consistency
        Assign candidate confidence
    end for
    Select earliest credible boundary satisfying the activity constraints
    Construct segment [b, c]
    if segment overlaps previously accepted segment then
        Reject or shift candidate
    end if
    Append accepted segment
    Update used_until
end for
Return all accepted segments
```

The choice of the *earliest credible boundary* is important. Selecting only the latest event before completion would recreate the short-segment problem observed in the earlier Dataset B version.

11 Complexity Analysis

Assuming the raw events are already chronologically ordered, a streaming implementation can process the event stream in approximately

$$O(N)$$

time for the basic feature extraction and boundary checks.

If each completion anchor searches a fixed maximum temporal window of bounded size, the practical cost remains close to linear in the number of events.

Memory can be approximately

$$O(N)$$

when all events are retained, or reduced toward

$$O(W)$$

for a streaming implementation, where W is the maximum temporal window retained.

This is substantially cheaper than training and running a large neural model.

The current method therefore has a strong computational-efficiency advantage for relatively small event-log datasets.

12 Skip Connections and Number of Layers

There are no hidden layers or skip connections in the final system.

For a future Transformer model, residual connections would typically be used:

$$h^{(l+1)} = h^{(l)} + F(h^{(l)}),$$

followed by normalization.

A reasonable experimental starting point would be a small model, for example:

2–4

Transformer encoder blocks rather than a very deep architecture.

The reason to start small is the limited amount of labelled data. Increasing model depth without increasing the amount and diversity of training data would increase overfitting risk.

This is an important engineering point: model capacity should be determined by the amount and quality of supervision, not by the fact that a Transformer is available.

13 Automation and Safety

The segmentation system feeds an automation analysis layer rather than directly executing actions.

For each task type k:

$$N_k = \text{frequency},$$

$$T_k = \sum_i T_{ki},$$

and a heuristic opportunity score is computed from frequency, average duration, cumulative effort and confidence.

The scenario ROI calculation is:

$$Saving_k = 0.70 T_k.$$

The 70% factor is an assumption and is not a measured production saving.

The invoice prototype follows a stricter rule:

$$Decision = \begin{cases} APPROVE, & \text{evidence complete and consistent} \\ REVIEW, & \text{otherwise.} \end{cases}$$

This prevents the automation layer from converting uncertainty into an irreversible action.

14 Efficiency: Current System vs Neural Network

The comparison should be made along several dimensions.

	Current system	**Potential neural system**
Training cost	None	Potentially significant
Inference cost	Low	Higher
GPU requirement	Not required	May be useful
Data requirement	Relatively low	Much higher
Interpretability	High	Lower
Boundary flexibility	Rule-limited	Learned from examples
Debugging	Direct	More difficult
Generalization	Limited by rules	Potentially better with enough data
Overfitting risk	Low from trainable parameters	Important concern
Deployment complexity	Low	Higher

Therefore the correct conclusion is not "neural networks are always better."

For the current dataset:

$$\boxed{\text{Rule-based baseline is computationally efficient and appropriate}}$$

For a larger labelled dataset:

$$\boxed{\text{A learned temporal model could improve boundary generalization}}$$

The strongest production design is likely a hybrid system.

15 Final Technical Takeaway

The project can be viewed as three layers:

$$\boxed{\text{Temporal segmentation} \rightarrow \text{Workflow understanding} \rightarrow \text{Safe automation}}$$

The current implementation solves the first layer using interpretable temporal reasoning.

A future AI version would learn the first layer:

$$\text{events} \rightarrow \text{temporal encoder} \rightarrow P(\text{boundary}) \rightarrow \text{segments}.$$

It is an algorithmic AI/ML-engineering baseline based on structured inference, temporal features and explicit decision rules.

The main limitation is not computational efficiency. It is that the rules cannot automatically learn new boundary patterns. That is exactly where a neural or other learned temporal model could become useful once sufficient labelled data is available.